

TiesVerse

Database Design Document

Version 1.1

June 2026

Custom Node/Express Backend

PostgreSQL / MySQL Compatible

Updated: Event Speakers Model

v1.1 Change Log — Event Speaker Model Revised

- Removed: Standalone speakers table (global reusable pool — not needed)
- Removed: Old event_speakers junction table (many-to-many link — not needed)
- Added: New event_speakers table — speakers are stored directly under each event with their own details
- Rule enforced: Team members are never speakers. Team members manage/organise events. Speakers are always external, unique per event.

Table of Contents

- Overview & Entity Relationship Summary
- Table: departments — team sub-groups
- Table: team_members — internal team profiles (organisers, not speakers)
- Table: team_member_socials — social links per member
- Table: events — online & offline events
- Table: event_speakers — external speakers per event (event-scoped, not reusable)
- Table: event_registrations — attendee form submissions
- Full DDL — CREATE TABLE Statements
- API Operations Reference
- Key Design Decisions

1. Overview & Entity Relationship Summary

The TiesVerse database supports internal team member profiles, event management (online & offline), event-specific speaker listings, and attendee registration. Images are stored externally (cloud storage) and referenced by URL. Registrants do not need user accounts — registration is a form submission only.

Key separation of concerns: Team members are the internal organising team — they manage events and represent TiesVerse. Speakers are always external guests, unique to each event. There is no shared pool of speakers across events; every event creates its own speaker records.

departments —< team_members > team_member_socials (organise/manage events, never speakers) events —< event_speakers (external speakers, stored per event, not reusable) —< event_registrations (attendee form submissions) Relationships: departments 1 — N team_members (one dept, many members) team_members 1 — N team_member_socials (one member, many social links) events 1 — N event_speakers (each event owns its speaker records) events 1 — N event_registrations (one event, many registrants)

Images: All `photo_url` and `cover_image_url` fields store a URL string pointing to files hosted on external cloud storage (e.g. AWS S3, Cloudinary). No binary image data is stored in the database.

2. Tables — Detailed Column Reference

01 departments

Lookup table for team sub-groups (e.g. Design, Tech, Outreach). Team members reference this table via a foreign key.

Column	Type	Constraints	Description
id	SERIAL	PK	Auto-incrementing primary key
name	VARCHAR(100)	UNIQUE NOT NULL	Department name, e.g. "Design Team"
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

02 team_members

Profiles for the internal TiesVerse team. These members **organise and manage events** — they are never listed as speakers. Supports soft-deletion via `is_active` so alumni records are preserved without appearing publicly.

Rule: No team member should ever appear in `event_speakers`. The two tables are intentionally separate and serve different purposes.

Column	Type	Constraints	Description
id	SERIAL	PK	Auto-incrementing primary key
name	VARCHAR(150)	NOT NULL	Full name
role	VARCHAR(150)	NOT NULL	Internal role, e.g. "Lead Designer", "Event Coordinator"
department_id	INT	FK → departments NULL	Which department this member belongs to
bio	TEXT	NULL	Short biography paragraph
photo_url	VARCHAR(500)	NULL	External URL to profile photo
is_active	BOOLEAN	DEFAULT TRUE	FALSE hides the member (alumni soft-delete)
display_order	INT	DEFAULT 0	Manual sort order for frontend rendering
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMP	DEFAULT NOW()	Last update timestamp

03 team_member_socials

Stores one or more social media links per team member. The `UNIQUE(team_member_id, platform)` constraint ensures each member has at most one link per platform.

Column	Type	Constraints	Description
id	SERIAL	PK	Auto-incrementing primary key
team_member_id	INT	FK → team_members CASCADE	Owner member; row deleted if member is deleted
platform	VARCHAR(50)	NOT NULL	e.g. 'linkedin', 'twitter', 'github', 'website'
url	VARCHAR(500)	NOT NULL	Full URL to the social profile

Unique constraint on `(team_member_id, platform)` — one link per platform per person

04 events

Central table for all TiesVerse events — both online and offline. Capacity is optional; NULL means unlimited. The `is_published` flag controls visibility on the public site.

Column	Type	Constraints	Description
<code>id</code>	<code>SERIAL</code>	<code>PK</code>	Auto-incrementing primary key
<code>title</code>	<code>VARCHAR(255)</code>	NOT NULL	Event title
<code>description</code>	<code>TEXT</code>	<code>NULL</code>	Full event description / agenda
<code>event_type</code>	<code>VARCHAR(20)</code>	CHECK IN ('online','offline')	Event format
<code>status</code>	<code>VARCHAR(20)</code>	DEFAULT 'upcoming'	'upcoming' 'ongoing' 'completed' 'cancelled'
<code>cover_image_url</code>	<code>VARCHAR(500)</code>	<code>NULL</code>	External URL to event cover/banner image
<code>venue_name</code>	<code>VARCHAR(255)</code>	<code>NULL</code> (offline only)	Name of the physical venue
<code>venue_address</code>	<code>TEXT</code>	<code>NULL</code> (offline only)	Full street address
<code>meeting_link</code>	<code>VARCHAR(500)</code>	<code>NULL</code> (online only)	Zoom / Meet / Teams URL
<code>start_datetime</code>	<code>TIMESTAMP</code>	NOT NULL	Event start date and time
<code>end_datetime</code>	<code>TIMESTAMP</code>	<code>NULL</code>	Event end date and time
<code>registration_deadline</code>	<code>TIMESTAMP</code>	<code>NULL</code>	Last date/time to register; NULL = no deadline
<code>capacity</code>	<code>INT</code>	<code>NULL</code>	Max attendees; NULL = unlimited
<code>is_published</code>	<code>BOOLEAN</code>	DEFAULT FALSE	FALSE = draft, hidden from public site
<code>created_at</code>	<code>TIMESTAMP</code>	DEFAULT NOW()	Record creation timestamp
<code>updated_at</code>	<code>TIMESTAMP</code>	DEFAULT NOW()	Last update timestamp

05 event_speakers

Stores speaker details directly for each event. Speakers are **always external guests** — never team members — and are **unique per event**. There is no shared speaker pool; each event creates its own speaker records from scratch. Both online and offline events use this same table.

Design note: This is a direct parent-child relationship (events → event_speakers), not a many-to-many junction. If the same real-world person speaks at two different events, they get two separate rows — one per event. This keeps speaker profiles accurate to each event context (title, bio, photo may differ).

Column	Type	Constraints	Description
<code>id</code>	<code>SERIAL</code>	<code>PK</code>	Auto-incrementing primary key
<code>event_id</code>	<code>INT</code>	<code>FK</code> → events CASCADE	The event this speaker belongs to
<code>name</code>	<code>VARCHAR(150)</code>	NOT NULL	Speaker's full name
<code>designation</code>	<code>VARCHAR(150)</code>	<code>NULL</code>	Title + company, e.g. "CTO at Acme Corp"
<code>organization</code>	<code>VARCHAR(150)</code>	<code>NULL</code>	Company or institution name
<code>bio</code>	<code>TEXT</code>	<code>NULL</code>	Short biography paragraph for this event
<code>photo_url</code>	<code>VARCHAR(500)</code>	<code>NULL</code>	External URL to speaker photo
<code>display_order</code>	<code>INT</code>	DEFAULT 0	Sort order of speakers on the event detail page
<code>created_at</code>	<code>TIMESTAMP</code>	DEFAULT NOW()	Record creation timestamp

06 event_registrations

Stores attendee form submissions for events. No user account is required — registrants submit their details only. The `UNIQUE(event_id, email)` constraint prevents duplicate registrations per event.

Column	Type	Constraints	Description
id	SERIAL	PK	Auto-incrementing primary key
event_id	INT	FK → events CASCADE	The event being registered for
first_name	VARCHAR(100)	NOT NULL	Registrant's first name
last_name	VARCHAR(100)	NOT NULL	Registrant's last name
email	VARCHAR(255)	NOT NULL	Registrant's email address
phone	VARCHAR(30)	NULL	Optional contact number
organization	VARCHAR(150)	NULL	Optional company / college name
status	VARCHAR(20)	DEFAULT 'confirmed'	'confirmed' 'waitlisted' 'cancelled' 'attended'
notes	TEXT	NULL	Admin notes or custom form responses
registered_at	TIMESTAMP	DEFAULT NOW()	Submission timestamp

Unique constraint on `(event_id, email)` — one registration per email per event

8. Full DDL — CREATE TABLE Statements

```
-- -----
-- TiesVerse Database Schema | v1.1 | June 2026
-- Compatible with PostgreSQL and MySQL (minor adjustments)
--
-- RULE: team_members = internal organisers. Never speakers.
--       event_speakers = external guests, unique per event.
-- -----

-- 1. DEPARTMENTS
CREATE TABLE departments (
  id          SERIAL          PRIMARY KEY,
  name        VARCHAR(100)    NOT NULL UNIQUE,
  created_at  TIMESTAMP       DEFAULT NOW()
);

-- 2. TEAM MEMBERS (internal only – organisers, not speakers)
CREATE TABLE team_members (
  id          SERIAL          PRIMARY KEY,
  name        VARCHAR(150)    NOT NULL,
  role        VARCHAR(150)    NOT NULL,
  department_id INT          REFERENCES departments(id) ON DELETE SET NULL,
  bio         TEXT,
  photo_url   VARCHAR(500),
  is_active   BOOLEAN         DEFAULT TRUE,
  display_order INT          DEFAULT 0,
  created_at  TIMESTAMP       DEFAULT NOW(),
  updated_at  TIMESTAMP       DEFAULT NOW()
);

-- 3. TEAM MEMBER SOCIALS
CREATE TABLE team_member_socials (
  id          SERIAL          PRIMARY KEY,
  team_member_id INT          NOT NULL REFERENCES team_members(id) ON DELETE CASCADE,
  platform    VARCHAR(50)     NOT NULL,
  url         VARCHAR(500)    NOT NULL,
  UNIQUE (team_member_id, platform)
);

-- 4. EVENTS (online and offline)
```

```

CREATE TABLE events (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  event_type VARCHAR(20) NOT NULL CHECK (event_type IN ('online', 'offline')),
  status VARCHAR(20) DEFAULT 'upcoming'
  CHECK (status IN ('upcoming', 'ongoing', 'completed', 'cancelled')),

  cover_image_url VARCHAR(500),
  venue_name VARCHAR(255), -- offline only
  venue_address TEXT, -- offline only
  meeting_link VARCHAR(500), -- online only
  start_datetime TIMESTAMP NOT NULL,
  end_datetime TIMESTAMP,
  registration_deadline TIMESTAMP,
  capacity INT, -- NULL = unlimited
  is_published BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- 5. EVENT SPEAKERS (external guests, scoped per event – not a reusable pool)
-- team_members must NEVER be inserted here.
CREATE TABLE event_speakers (
  id SERIAL PRIMARY KEY,
  event_id INT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  name VARCHAR(150) NOT NULL,
  designation VARCHAR(150),
  organization VARCHAR(150),
  bio TEXT,
  photo_url VARCHAR(500),
  display_order INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

-- 6. EVENT REGISTRATIONS
CREATE TABLE event_registrations (
  id SERIAL PRIMARY KEY,
  event_id INT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL,
  phone VARCHAR(30),
  organization VARCHAR(150),
  status VARCHAR(20) DEFAULT 'confirmed'
  CHECK (status IN ('confirmed', 'waitlisted', 'cancelled', 'attended')),

  notes TEXT,
  registered_at TIMESTAMP DEFAULT NOW(),
  UNIQUE (event_id, email)
);

```

9. API Operations Reference

Team Members

```
GET /api/team-members List all active team members (WHERE is_active = TRUE)

GET /api/team-members/:id Get a single team member with their social links

POST /api/team-members INSERT a new team member

PUT /api/team-members/:id UPDATE team member fields

POST /api/team-members/:id/socials INSERT or upsert a social link for a member

DELETE /api/team-members/:id/socials/:platform Remove a specific social link by platform name
```

Events

```
GET /api/events List all published events (WHERE is_published = TRUE)

GET /api/events/:id Get event detail including its speakers list

POST /api/events INSERT a new event (admin only)

PUT /api/events/:id UPDATE event fields including status and capacity
```

Event Speakers (scoped to a specific event)

```
GET /api/events/:id/speakers List all speakers for a specific event

POST /api/events/:id/speakers Add a new speaker to this event (INSERT into event_speakers)

PUT /api/events/:id/speakers/:sid UPDATE a speaker's details for this event

DELETE /api/events/:id/speakers/:sid Remove a speaker from this event
```

Registrations

```
POST /api/events/:id/register Submit registration – checks capacity before INSERT

GET /api/events/:id/registrations List all registrants for an event (admin only)

PUT /api/events/:id/registrations/:rid UPDATE registration status (e.g. mark as 'attended')

DELETE /api/events/:id/registrations/:rid Cancel a registration (UPDATE status = 'cancelled')
```

10. Key Design Decisions

Decision	Rationale
Team members are never speakers	Team members are the internal organising team for TiesVerse. Speakers are always external guests brought in per event. Keeping them in separate, unlinked tables enforces this at the data level.
Speakers are event-scoped (no reusable pool)	Speakers are different for every event. Storing them directly under each event (1-to-many) avoids a redundant global pool and a junction table. A speaker's bio or photo may also differ per event context.
<code>capacity</code> is nullable on events	Some events are unlimited. A <code>NULL</code> value means no cap. The API checks <code>COUNT(registrations) < capacity</code> before allowing a new INSERT.
<code>status = 'waitlisted'</code> on registrations	When an event is at capacity, registrants can still be collected as waitlisted instead of being rejected outright.
Separate <code>team_member_socials</code> table	Avoids cramming multiple nullable columns (<code>linkedin_url</code> , <code>twitter_url</code> , ...) onto the team member row. Easy to add new platforms without a schema migration.
<code>is_active</code> on <code>team_members</code> (soft-delete)	Alumni records are hidden from the public page but preserved for historical reference.
<code>is_published</code> on events	Events can be created and edited as drafts before going live. The public API only returns rows where <code>is_published = TRUE</code> .
<code>display_order</code> on <code>team_members</code> and <code>event_speakers</code>	Gives the admin manual control over rendering order without relying on insertion order or <code>id</code> values.
Images stored as URLs	Binary blobs bloat the DB and slow queries. Cloud storage handles CDN, resizing, and caching. The DB stores only a <code>VARCHAR</code> URL string.
No user accounts for registrants	Reduces friction. Registration is a one-time form submission. Email uniqueness per event prevents duplicate signups.